

Predictive Decision-Support Framework for Sustainable On-Premises Data Center Resource Optimization

AKILADEVI R

*Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
akiladevi.r@rajalakshmi.edu.in*

KARTHIKHA SRE M

*Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701143@rajalakshmi.edu.in*

KAVIYA MADHIRAJU

*Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701148@rajalakshmi.edu.in*

AISHWARYA A

*Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701015@rajalakshmi.edu.in*

AISHWARYA M

*Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701016@rajalakshmi.edu.in*

Abstract— On-premises data centers today operate around the clock to guarantee availability but a large share of the electricity, cooling capacity and water they have to consume goes toward servers that are barely being used. This stays hidden because the telemetry that could reveal it are typically scattered across disconnected monitoring tools that were never built to talk to each other. Our work addresses this gap through GreenOps, an observability and resource intelligence platform that unifies this data in one integrated pipeline and makes it intelligible to human operators. The system collects live telemetry (simulated in this prototype) from server, power, cooling and storage monitors and stores it with full historical context and computes real time sustainability metrics including Power Usage Effectiveness (PUE), Water Usage Effectiveness (WUE), energy cost and carbon emissions. GreenOps employs four machine learning models - CPU estimation, power estimation, workload forecasting and memory workload forecasting each trained from related telemetry. For every model, three candidate algorithms (Linear Regression, Random Forest and XGBoost) are evaluated honestly on held out data giving operators both a real-time and a near-future predictive view. Idle servers, stale storage and over-provisioned resources get flagged by a rule-based detection layer. If a server is flagged idle, a forecasting layer projects its near term and a what-if simulation engine trials possible migrations and estimates the savings involved. A priority-weighted recommendation engine then ranks these against cost, carbon and performance risk. A React based dashboard surfaces all of this, built for a human to review rather than for automation to act on unsupervised. This paper describes the system's architecture, the reasoning behind the modeling choices and results from our working prototype.

Keywords— *GreenOps, On-premises Data Centers, Sustainability, Machine Learning, Resource Optimization, Workload Forecasting, PUE, WUE*

I. INTRODUCTION

Data Centers have quietly become one of the most power-hungry categories of infrastructure in the modern

world. Whether a website is being loaded, a model is being trained, or a file is being synced to the cloud, it all traces back to a room full of servers running continuously throughout the day, regardless of whether anyone is actually using them at that moment. This 24/7 operation is by design, as availability is the goal, but it occurs at a steep environmental cost. The facilities that support this architecture also carry enormous overhead in the form of cooling systems, backup power infrastructure and water usage all of which is driven by how much hardware is kept running rather than by how useful the work it performs is.

A significant portion of this consumption is completely avoidable. Servers frequently sit at low utilization for long periods of time, often because they were sized for a peak that rarely arrives, or because the workload they were built for moved on and nobody switched them off. In both cases, they keep drawing power, taking up rack space, and generating heat that has to be cooled. This waste is difficult for operators to detect, and what goes undetected is rarely corrected.

Traditional monitoring tools tell an administrator what a server is doing right now, but rarely surface whether that usage pattern justifies that resources being spent on it and almost never translates that pattern into an actionable recommendation.

A 2026 CEEW-SYSTEMIQ study puts India's data center capacity at roughly 1.5 GW, up nearly threefold since 2020 and headed toward 6.5 GW by 2030, already consuming close to 150 billion litres of water a year [1]. Yet most Indian states with data center policies don't even require PUE or WUE reporting, while the EU, Germany, China and Singapore all enforce some form of it [1]. That gap is the real problem here without a reporting requirement, the kind of waste that can sit unnoticed for years, largely because nothing in day-to-day operations forces anyone to go looking for it.

II. LITERATURE SURVEY

A. Energy-Aware Resource Allocation and Server Consolidation

Much of the early work on data center efficiency problems was allocation. Deciding where workloads should run so that fewer machines carry more of the load. Beloglazov and Buyya[2] proposed energy-aware heuristics for virtual machine placement and migration and showed that dynamic consolidation reduces power draw substantially compared to static allocation, though the system acts autonomously with no provision for human review before a decision is executed.

Subramanian et al. [3] approached the same problem with more mathematical precision, modeling server consolidation as a variant of variable-sized bin packing, proving it NP-hard and supplying an approximation algorithm with bounded guarantees, the tradeoff is an assumption of known, stable demand that rarely holds in a data center where workloads shift hour to hour.

Two later studies complicate the picture usefully. [4] Lovasz et al. examined what consolidation costs in performance rather than only what it saves in energy. A tradeoff much of the purely energy-focused literature leaves unexamined.

[5] Uddin et al. then gave that tradeoff its concrete numbers, providing big drops in both power draw and total ownership cost after combining an enterprise server fleet, even though their study is a single retrospective audit rather than a system built to keep finding new inefficiencies as they happen.

B. Machine Learning-Based Workload and Resource Prediction

A more recent line of work uses machine learning to forecast resource use in advance, rather than optimizing allocation after the fact. [6] Bawa et al. built a two-stage ensemble with XGBoost and LightGBM. It first classifies whether a workload is idle or active, then predicts CPU and memory utilization for it.

[7] Deva and Jain took a similar approach, pairing Azure AutoML with XGBoost to predict cloud resource utilization in an enterprise setting, and they reported strong accuracy.

[8] Khan et al. went a step further by including provisioned resources, disk throughput and network metrics, which earlier prediction work mostly left out even though they clearly affect host-level energy use. One number from their study is worth citing directly, CPU and memory utilization in Google's own production clusters rarely went past 60%. That is exactly the kind of underutilization these systems are built to catch. Because measuring energy consumption at the level of individual virtual machines is difficult in practice, the same authors also proposed grouping machines by energy-relevant behaviour through clustering rather than instrumenting each one directly.

C. Carbon-Aware Scheduling and Cooling Optimization

[9] Radovanovic et al. describe a system used within Google that shifts flexible workloads across time and

geography according to forecasted grid carbon intensity, cutting operational carbon meaningfully, though the approach depends on multi-region infrastructure and workload portability that exist mainly at hyperscale. Cooling receives its own dedicated treatment elsewhere.

[10] Zhan et al. trained a physics-informed offline reinforcement learning system on historical data specifically to avoid the risk of experimenting directly on a live cooling system and Alex et al. [11] combined reinforcement with agglomerative clustering to guide VM placement according to a carbon-efficiency score computed per data center. Neither reinforcement learning approach yet has been validated on production infrastructure. Both remain simulation results, promising but unproven at scale.

D. Sustainability Metrics, Policy and ESG Reporting

A different strand of the literature is less concerned with how to optimize a data center and more concerned with whether anyone is required to measure it all. [12]Grochulska-Salak et al. argue that PUE alone cannot characterize sustainability and propose a broader set of indicators spanning water use, waste heat recovery, renewable integration and land use. Regulation is beginning to catch up with that argument; the EU's Corporate Sustainability Reporting Directive in force since January 2025, now requires large enterprises to disclose detailed environmental metrics and California's Climate Corporate Data Accountability Act (SB 253) will require Scope 1 and 2 emissions reporting starting in 2026 [13].

[14] Tripathi et al. offer a look at how this plays out on the ground. India's data center capacity has nearly tripled since 2020, yet most states have no PUE, WUE or carbon standards in their policies, which suggests the constraint is regulatory rather than technical. A 2025 perspective piece on what data center growth means for public health [15] found noise levels inside some facilities above safe exposure thresholds, though it notes that the evidence on these impacts is still thin overall. A similar pattern holds for water and energy disclosure: reporting from the Lincoln Institute of Land Policy[16] finds that operators routinely withhold consumption figures from host communities behind non-disclosure agreements. The engineering literature reaches a comparable conclusion by a different route. [17] A broader 2025 IEEE survey argues that energy efficiency, cooling and newer approaches such as hyper-converged infrastructure are most effective when pursued jointly rather than in isolation, a view anticipated by earlier work from Ahmed et al.[18],who showed that neural-network based workload forecasting improves energy efficiency without a corresponding loss in service quality.

E. Research Gap and Positioning of GreenOps

Across this body of work, three distinct strands emerge and each stops short of what the next one starts. Systems that optimize placement act on their own, with no step where a person reviews the decision before it executes. Systems that predict resource use stop at the forecast itself, without going on to rank or justify a corrective action. Work on sustainability reporting and policy documents the absence of continuous,

standardized tracking but stops short of building anything to fill that gap. None of the systems reviewed here combine continuous ML-based prediction, rule-based waste detection and unified PUE/WUE/carbon reporting with a recommendation a human can actually review before anything changes. GreenOps is built at the point where these three strands meet.

III. METHODOLOGY

GreenOps is built as a continuous pipeline from raw telemetry to a human reviewed recommendation. No stage acts on infrastructure without explicit approval, unlike the optimization systems in Section II. Every stage writes back to a shared database for future reference. The system is organized into four functional modules, shown in Fig 1.

A. System Architecture

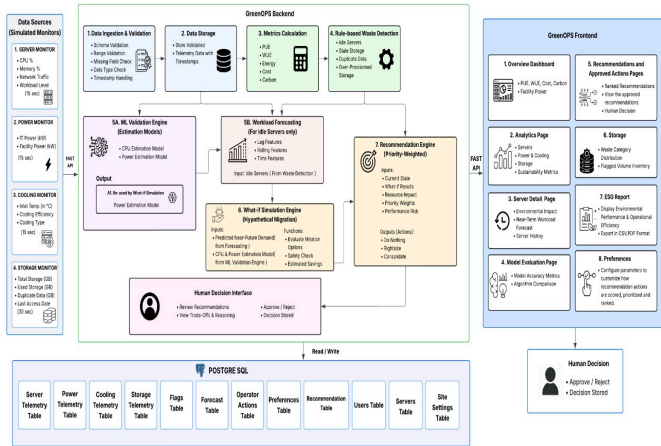


Fig. 1. GreenOps Architecture diagram

Fig 1. Shows the complete GreenOps architecture, spanning four layers: data source, backend processing, a shared persistence layer and a frontend dashboard. On the data-source side, four monitors, generate readings every 15 to 30 seconds - a Server Monitor tracking CPU, memory, network traffic and workload level; a Power Monitor tracking IT power and facility power; a Cooling Monitor tracking inlet temperature, cooling efficiency and cooling type and a Storage Monitor tracking total and used storage, duplicate data and days since last access. In this prototype, these four monitors are simulated rather than connected to physical hardware, since the working environment did not provide access to a live data center or its real monitoring tools. The simulators were built to mirror the structure and behavior of real telemetry sources so that the rest of the pipeline from ingestion through to recommendation, operates exactly as it would against a live facility. In a production deployment, this same ingestion layer would connect to the actual monitoring APIs that the data center already runs without requiring changes further down the pipeline.

Telemetry reaches the backend through a FastAPI ingestion layer, where it is checked for schema conformity, valid ranges and missing fields before being timestamped and written to a PostgreSQL database. From there, the same data feeds a

metrics-calculation stage, a rule-based waste-detection stage and two machine learning stages; one estimating current CPU and power behavior and the other forecasting near-future demand for servers already flagged as idle. The forecasting output feeds a what-if simulation engine, which in turn feeds a priority-weighted recommendation engine. Every recommendation this engine produces is written to the database and surfaced through a React-based frontend, where an operator makes the final approve-or-reject decision through a human decision interface. Twelve distinct tables are in the shared database keeping every stage's inputs and outputs available to every other stage which is what allows recommendation engines for instance, to reason about a server's full history rather than only its most recent reading.

B. Module 1: Data Pipeline and Observability Layer

This module covers ingestion, storage, real-time metrics calculation and rule-based waste detection and its job is to turn raw, noisy telemetry into something the rest of the system can trust. The live incoming data is being simulated currently since there is no access to real telemetry, the simulation runs multiple servers across four types: four Compute, three GPU, three Storage and three Edge servers with a few servers in each type deliberately biased toward low utilization, so the recommendation engine always has more than one real candidate to choose from.

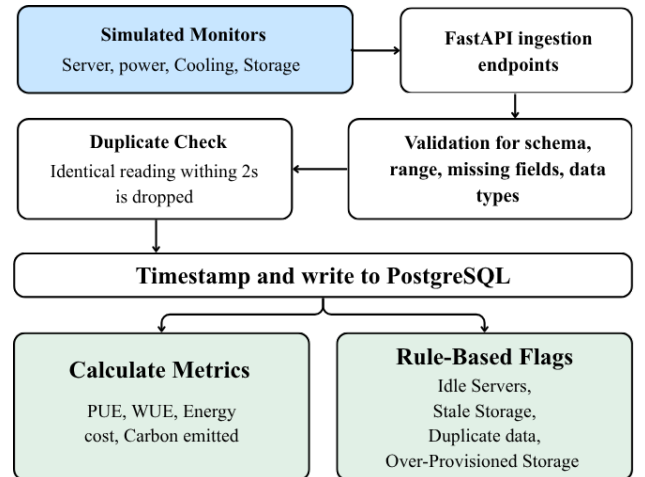


Fig. 2. Module 1 pipeline diagram

Fig 2. Shows how a reading gets from a simulated monitor to something the rest of the system can use. Server, power and cooling readings all post every 15 seconds. Storage posts every 30, since a volume's used space and duplicate count don't shift nearly as fast as a server's CPU does. Each reading goes through schema, range and type checks before it's timestamped and written to PostgreSQL and if an identical reading arrives within two seconds, it gets dropped so a retried delivery is never counted twice.

Once a reading clears validation it is sent to two places. It feeds the metrics calculation which produces PUE, WUE, energy cost and carbon emissions. Carbon uses a configurable grid emission factor defaulting to 0.710 kg/kWh based on

India's national grid average and it can be adjusted for a different site.

$$PUE = \text{Facility Power (kW)} / \text{IT Power (kW)}$$

Water Usage Effectiveness (WUE) is calculated as

$$WUE = \text{Total Water consumed (L)} / \text{IT Energy (kWh)}$$

Energy Cost is calculated as

$$\text{Energy Cost} = \text{Energy consumed} * \text{applied tariff}$$

Carbon Emissions are calculated as

$$\text{Carbon (kg)} = \text{Energy (kWh)} * 0.710 \text{ kg/kWh (a placeholder grid carbon intensity)}$$

The reading is also fed into the rule engine, which checks the same reading for four waste patterns: idle servers when CPU usage is less than a threshold for a time period, stale storage when data isn't being used in almost a year, duplicate data and over-provisioning where the resources used is lesser than the resource allocated.

C. Module 2: Predictive Intelligence Engine

This module covers CPU and power estimation, workload forecasting for idle servers and what-if simulation and its where GreenOps moves from describing the present state of the data center to reasoning about its near future. The three components are chained rather than independent: the estimation models establish what a server is doing right now, forecasting projects where an idle server is headed next and simulation uses both of these to test hypothetical changes before anything is recommended

1) CPU and Power Estimation:

The ML validation engine trains two related models rather than just one.

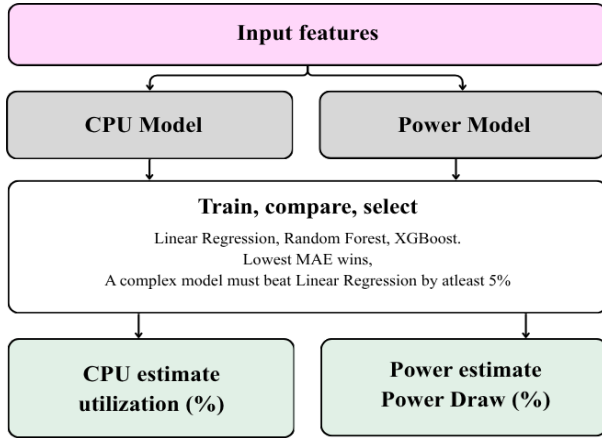


Fig. 3. CPU / Power pipeline diagram

Fig 3. Shows the two estimation models - Power model and CPU model side by side. Both take the same categorical and numerical telemetry, server type, cooling type, region, time of day, workload, memory, network, inlet temperature, cooling efficiency and PUE. The power model gets one extra input, the server's own current CPU reading since it tracks how hard a server is working. Both models are trained and compared the same way using Linear regression, Random Forest and XGBoost algorithms. These three are trained with an 80/20 train-test split, scored on Mean Absolute Error (MAE), Root Mean Squared Error (RMSE) and R². The lowest Mean

Absolute Error (MAE) wins and a complex model only gets chosen if it beats Linear Regression by more than 5%. On this dataset, Linear Regression won both times. A different environment could easily get Random Forest or XGBoost instead.

2) Workload Forecasting:

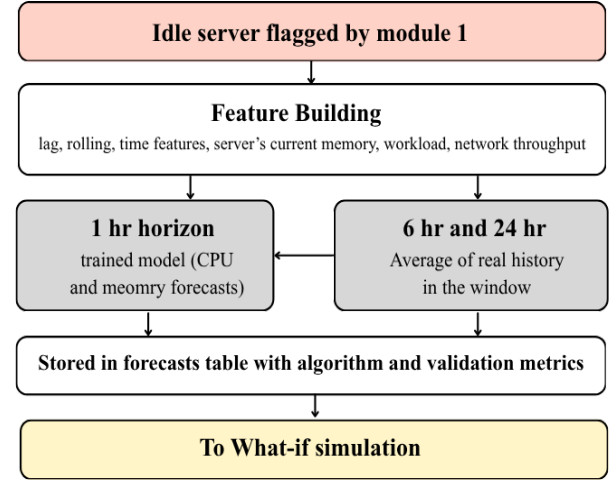


Fig. 4. Workload Forecasting pipeline diagram

Fig 4. Shows what happens to a server once Module 1 flags it as idle. GreenOps checks whether the server is genuinely underused or just between tasks right now. It does this by building three kinds of features from the server's own recent history: lag features for what it was doing a few readings ago, rolling features for short-term trends and time features for the daily rhythm most workloads follow. Its current memory, workload and network throughput are given as input too, so the model has both where the server has been and where it stands right now.

Two separate models come out of this, one for CPU demand and one for memory demand, each with its own file and its own MAE, RMSE and R² scores. This mirrors the CPU and power split, hence each target's accuracy and selected algorithm stay independently visible to an operator. Both forecasts feed the simulation engine when it evaluates a consolidation candidate's near-term demand for the server rather than reacting only to its most recent reading, which is what allows GreenOps to distinguish a server that is briefly quiet from the one that has been genuinely underutilized for an extended period.

At one hour out, there's enough real history to train a model properly. At six and twenty-four hours, a trained model would likely overshoot a server that's just holding steady. So GreenOps averages whatever real history actually exists in that window. If there isn't enough history yet, it says so instead of guessing. This helps keep the trust in this software since every number an operator sees traces back to something the system actually observed.

3) What-If Simulation:

This simulation gets the flagged server and the candidate server and asks: What would really happen if the server's workload was moved, combined or resized?

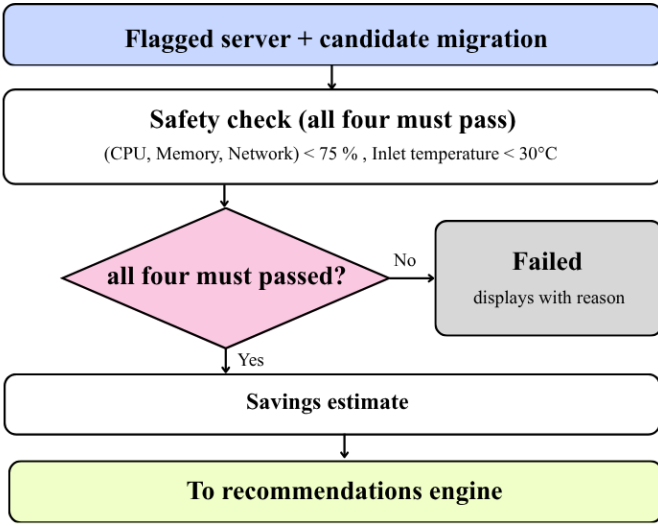


Fig. 5. What-if Simulation pipeline diagram

Fig 5. Shows the safety check every migration option has to pass. Before GreenOps estimates any savings, it asks whether moving a server’s workload is actually safe, not just whether it would save energy. A candidate has to clear four checks at once: CPU under 75%, memory under 75%, network throughput under 75% of link capacity and a predicted inlet temperature under 30°C after the move. CPU and memory get checked against both the current reading and the forecast, while network and thermal are judged on the current values only. A candidate only becomes the recommended target if it clears both the present-moment and the near-future check. But failing doesn’t mean disappearing. A candidate that fails just the forecast, the network limit or the thermal check still shows up to the operator with a specific reason attached.

Once a candidate clears all four checks, the engine estimates what the move would actually save:

$$\begin{aligned}
 \text{energy_saved} &= \max(0, \text{energy_before} - \text{energy_after}) \\
 \text{carbon_saved} &= \text{energy_saved} \times 0.710 \text{ kg/kWh} \\
 \text{cost_saved} &= \text{energy_saved} \times ₹8/\text{kWh} \\
 \text{water_saved} &= \text{energy_saved} \times \text{WUE_FACTOR} \\
 &\quad [\text{source's cooling type}]
 \end{aligned}$$

Energy_after comes from the power model’s prediction on the candidate’s full post-move profile, scaled to facility power via its own PUE, with a simple CPU-based estimate used as a fallback when the model or cooling data isn’t available.

D. Module 3: Decision Support and Human Interface

This module covers recommendation ranking, human review and the frontend dashboard and it is the layer where the prediction and simulations are turned into something an operator can actually act on. A priority-weighted recommendation engine combines four inputs - the server’s current state, the what-if simulation’s results, the estimated resource impact of each option and a set of configurable priority weights reflecting what the organization currently cares about most whether that is cost, carbon or performance risk, to rank pending recommendations against each other. Each recommendation already carries a fixed action type

determined by its originating flag such as rightsizing, consolidation, archival or deduplication and the operator then applies one of three decisions to any given recommendation: accept it, snooze it for re-evaluation or dismiss it. This ranking is computed as weighted sum of normalized savings, minus a penalty for risk:

$$\begin{aligned}
 \text{Score} &= w_{\text{energy}} \cdot \text{norm}(\text{energy_saved}) + \\
 &w_{\text{cost}} \cdot \text{norm}(\text{cost_saved}) + w_{\text{carbon}} \cdot \text{norm}(\text{carbon_saved}) + \\
 &w_{\text{water}} \cdot \text{norm}(\text{water_saved}) - w_{\text{risk}} \cdot \text{risk_score}
 \end{aligned}$$

Each value is normalized against the largest of its kind on the current batch, so raw units never dominate the score. Weights are stored per organization and editable live from the Preferences page, re-ranking recommendations instantly with no retraining. Rather than applying any of these automatically, the system writes each ranked recommendation to the database and surfaces it through the frontend and the IT infrastructure manager decides whether the recommendations are approved. Then the appropriate action is being performed meaning that no infrastructure change in GreenOps ever happened without a human decision behind it, the approved actions can also be viewed in a separate page to keep track of the actions. Every decision in GreenOps is also tied to who is allowed to make it. The system defines three roles: an infrastructure manager who can review, accept, snooze or reject recommendations and manage other users; a sustainability manager who has read-only access to the dashboards and ESG reports but cannot act on anything and an operations engineer who can mark an already-approved action as executed but cannot make the original accept-or-reject decision. This split helps differentiate roles of who is allowed to make the call, while execution and oversight are kept to separate roles. The thirteen-page React dashboard gives a complete overview of everything necessary for the users which is discussed in detail in Section IV.

E. Module 4: ESG Reporting

Alongside the operator-facing recommendation workflow, GreenOps maintains a dedicated ESG reporting layer that aggregates the telemetry into a facility-level sustainability rather than a per-decision view. This does not wait on human approval. Energy, carbon, cost, water, WUE, average PUE, average CPU and memory load, cooling efficiency and the count of underutilized servers are all calculated directly from raw telemetry over a selected date range, whether or not an operator has acted on anything during that window. A separate section of the same report tracks the decision side: how many recommendations were accepted, snoozed or dismissed and what the accepted ones were estimated to save in energy, carbon, cost and storage at the moment each was approved. Reports are downloadable in both CSV and PDF form.

IV. IMPLEMENTATION

The methodology described in Section III was implemented as a working prototype, covering the full pipeline from telemetry ingestion through to the human decision interface.

A. Backend Implementation

The backend is implemented in FastAPI with SQLAlchemy over PostgreSQL, exposing telemetry ingestion, analytics, forecasting, recommendation and ESG reporting endpoints

Each of the four simulated monitors post to its own ingestion endpoint, where they are validated before being written to their respective telemetry table.

TABLE I. DATABASE BACKEND

No	Table	Purpose
1.	Servers	Server registry - static identity / metadata for each of the 13 simulated servers
2.	Server telemetry	Raw compute readings, posted every 15s
3.	Power telemetry	Raw power readings, posted every 15s
4.	Cooling telemetry	Raw cooling readings, posted every 15s
5.	Storage telemetry	Raw storage readings posted every 30s
6.	Forecasts	Persisted CPU forecasts (trained model at one hour, historical average at six and twenty four hours) with each prediction's algorithm and validation metrics.
7.	Flags	Active / cleared rule based waste detection (idle, stale, duplicate, over provisioned)
8.	Recommendations	Ranked actions generated from a flag, decision-support only
9.	Operator actions	The operator's decision and the what-if impact estimate at that exact moment
10.	Preferences	Single-row, live editable scoring weights used by the recommendation ranking formula
11.	Users	To store Role based access and sign in accordingly.
12.	Site Settings	Single-row, operator-editable config — grid emission factor plus declared values telemetry can't derive (installed capacity, renewable/reuse factors)

B. Frontend Implementation

The frontend is a React single-page application with thirteen routed pages.

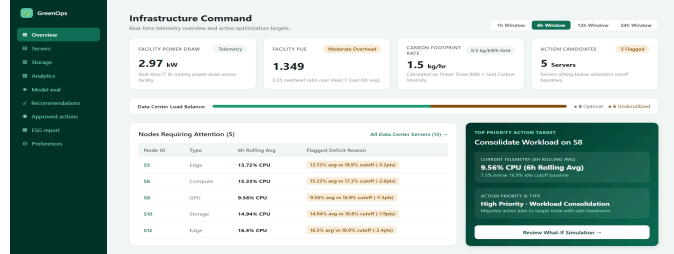


Fig. 6. Dashboard Overview page

The Overview page summarizes facility wide values for PUE, cumulative energy, carbon and the current count of flagged servers being refreshed from live simulated telemetry.

The Servers page shows the full server inventory filterable by type, region and status (Healthy/Underutilized/Stale/No Data), giving an operator a single screen to scan everything before drilling into any one machine. The Server detail page gives insights into per-server view combining live environmental reading, the CPU forecast and any active waste flags for that specific server.

The Storage page shows specific waste view, surfacing stale, duplicated and over-provisioned volumes flagged by the rule engine.

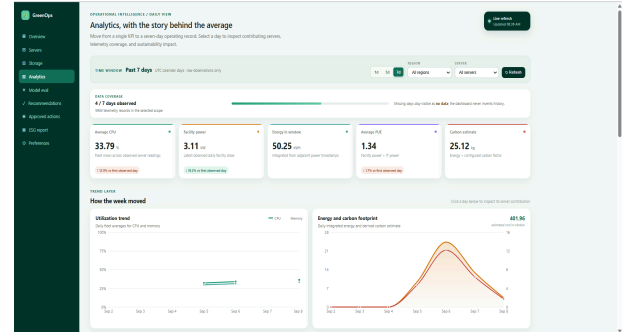


Fig. 7. Analytics page

The Analytics page breaks down the facility-level of idle servers, power/cooling performance and storage waste over time which is used to spot trends.

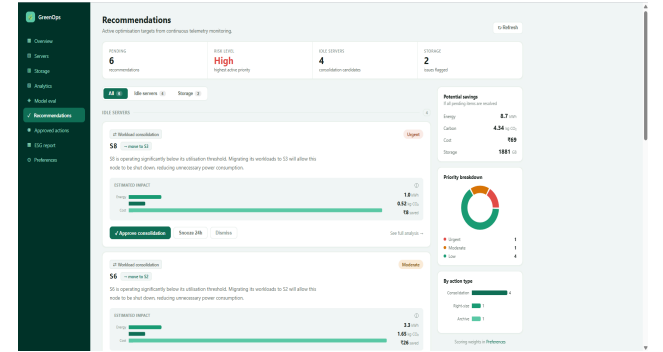


Fig. 8. Recommendations page

The Recommendations page ranks a list of pending consolidate/rightsize actions with inline Accept / Snooze / Dismiss controls, mini impact charts and a sidebar summarizing pending savings across the batch.

The Recommendation details page gives full insights about a single recommendation. It ranks all the candidate tables with a CPU history chart, a storage breakdown and the ability to switch candidates and see the impact estimate re-scale live.

The Approved actions logs the past operators decisions, filterable by executed vs awaiting executions where an operator can retroactively attach an executed at timestamp and a note once the action is carried out.

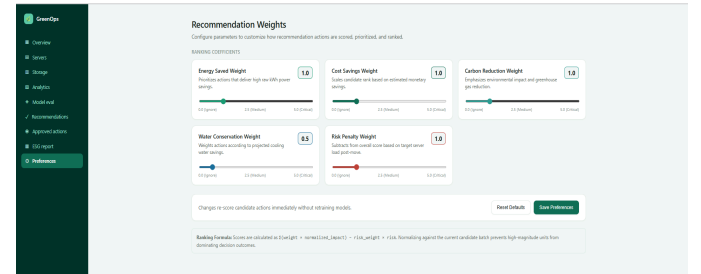


Fig. 9. Preferences page

The Preferences page has a weight slider for cost, carbon, water and risk that feeds directly into the recommendation scoring formula, re-ranking pending recommendations instantly without retraining any model.

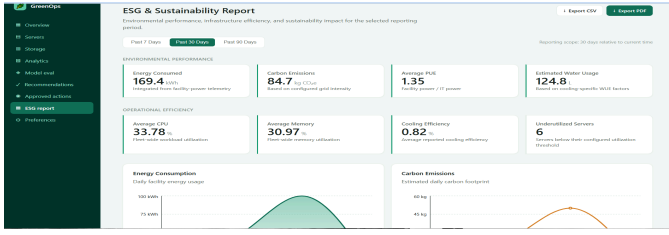


Fig. 10. ESG reporting page

This page has aggregated log of operator-action over a selected date range, exportable as CSV or PDF, presented explicitly as a decision record.

V. RESULTS AND EVALUATION

A. CPU and Power Estimation

Both estimation models were trained on the Kaggle “Green AI Data Center Telemetry” dataset (10,000 records and 15 columns), split by 80/20 with a most-frequent category imputer handling missing values in data center region (3,990 missing entries). The selection rule kept Linear Regression as the default and only promoted Random Forest or XGBoost if either beat it by more than 5% relative MAE.

TABLE II. CPU ESTIMATION

Model	MAE (%)	RMSE (%)	R^2
Linear Regression	4.511	7.671	0.845
Random Forest	4.828	8.077	0.828
XGBoost	4.833	7.999	0.831

TABLE III. POWER ESTIMATION

Model	MAE (kW)	RMSE (kW)	R^2
Linear Regression	0.211	0.330	0.422
Random Forest	0.227	0.346	0.366
XGBoost	0.222	0.341	0.385

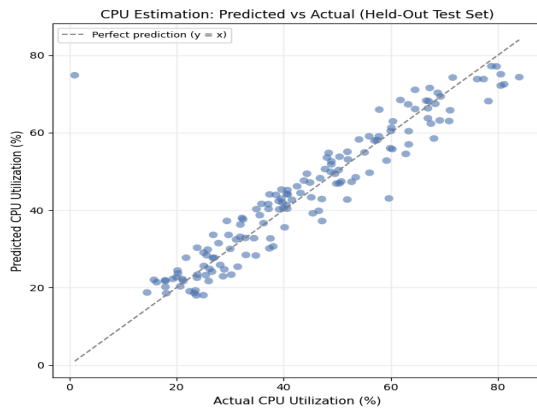


Fig. 11. CPU Estimation: Predicted vs Actual

Fig.11 demonstrates the practical accuracy of our model, where an R^2 value of 0.845 shows a strong alignment between predicted and actual CPU utilization. The data points consistently follow the ideal prediction line, with minor variance isolated to the extreme values where training samples were naturally sparse.

B. Workload Forecasting

The forecasting models were trained separately using telemetry accumulated by the live simulator.

TABLE IV. CPU WORKLOAD FORECAST

Model	MAE (%)	RMSE (%)	R^2
Linear Regression	6.514	8.090	0.886
Random Forest	6.703	8.392	0.877
XGBoost	6.567	8.178	0.883

TABLE V. MEMORY WORKLOAD FORECAST

Model	MAE (%)	RMSE (%)	R^2
Linear Regression	6.560	8.185	0.851
Random Forest	6.843	8.527	0.839
XGBoost	6.795	8.489	0.840

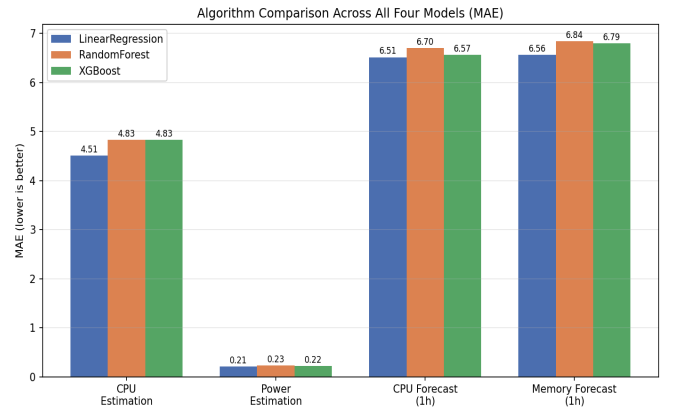


Fig. 12. Comparing algorithms across all four models using MAE

Across all three algorithms the margin between Linear Regression and its more complex alternatives was under 5% MAE in every case so the simpler model was kept throughout as shown in Fig. 12.

VI. CONCLUSION AND FUTURE WORK

GreenOps demonstrates that the capabilities most of the reviewed literature treats separately with continuous observability, ML-based prediction, rule-based waste detection and sustainability reporting can run as one pipeline with a single human decision point at the end. The working prototype validates this end to end with estimation models trained and honestly evaluated on real telemetry, forecasting models trained on the system’s own accumulated history, a four-dimension what-if safety check that screens every migration option before it’s ever surfaced and role-based

review that ties each decision to a specific accountable person rather than to automation.

Future work will focus on replacing the simulated telemetry with real integrations (Prometheus, smart PDU's/BMS, API's) and swapping the placeholder carbon-intensity and WUE constants for live regional grid and facility water data. The what-if check should also gain workload criticality gating, along with SLA requirements and storage and electrical capacity limits, so that no recommendation is surfaced for a workload that is too critical to move. Longer-range planning needs a 24-hour forecast. That's not possible yet, since there isn't enough telemetry to train it on. Nothing currently checks whether an approved migration saved what was predicted either, so a before-and-after comparison of power readings is worth adding. Beyond that, GreenOps could be tried across several data centers and watched over a longer period, to see whether operators come to trust it as its track record grows.

VII. REFERENCES

- [1] Council on Energy, Environment and Water (CEEW), "How is data centre infrastructure in India shaping power and water use?," 2026.
- [2] A. Beloglazov, J. Abawajy and R. Buyya, "Energy-aware resource allocation heuristics for efficient management of data centers for cloud computing," *Future Generation Computer Systems*, vol. 28, no. 5, pp. 755–768, 2012.
- [3] C. Subramanian, A. Vasan and A. Sivasubramaniam, "Reducing data center power with server consolidation: Approximation and evaluation," in *Proc. 17th Int. Conf. High Performance Computing (HiPC)*, IEEE Computer Society, 2010.
- [4] G. Lovász, F. Niedermeier and H. de Meer, "Performance tradeoffs of energy-aware virtual machine consolidation," *Cluster Computing*, vol. 16, no. 3, pp. 481–496, 2013.
- [5] M. Uddin *et al.*, "Server consolidation: A technique to enhance cloud data center power efficiency and overall cost of ownership," *International Journal of Distributed Sensor Networks*, 2021.
- [6] J. Bawa, K. K. Chahal and K. Kaur, "Improving cloud resource management: An ensemble learning approach for workload prediction," *The Journal of Supercomputing*, vol. 81, no. 10, 2025.
- [7] M. R. R. Deva and N. Jain, "Utilizing Azure Automated Machine Learning and XGBoost for predicting cloud resource utilization in enterprise environments," in *Proc. 2025 Int. Conf. Networks and Cryptology (NETCRYPT)*, IEEE, 2025.
- [8] T. Khan, W. Tian, S. Ilager and R. Buyya, "Workload forecasting and energy state estimation in cloud data centres: ML-centric approach," *Future Generation Computer Systems*, vol. 128, pp. 320–332, 2022.
- [9] A. Radovanović *et al.*, "Carbon-aware computing for datacenters," arXiv preprint arXiv:2106.11750, 2021.
- [10] X. Zhan *et al.*, "Data center cooling system optimization using offline reinforcement learning," arXiv preprint arXiv:2501.15085, 2025.
- [11] M. Alex, S. O. Ojo and F. M. Awuor, "Carbon-aware, energy-efficient and SLA-compliant virtual machine placement in cloud data centers using deep Q-networks and agglomerative clustering," *Computers*, vol. 14, no. 7, Art. no. 280, 2025.
- [12] M. Grochulska-Salak *et al.*, "Green data centres: Sustainable solutions with green energy and green-blue infrastructure," *Energies*, vol. 18, no. 24, Art. no. 6592, 2025.
- [13] European Commission, "Corporate sustainability reporting," Financial Markets, Company Reporting and Auditing.
- [14] Council on Energy, Environment and Water (CEEW) and SYSTEMIQ, "Scaling India's data centre ecosystem," 2026.
- [15] Y. Tao and P. Gao, "Global data center expansion and human health: A call for empirical research," *Eco-Environment & Health*, vol. 4, no. 3, Art. no. 100157, 2025.
- [16] J. Gorey, "Data drain: The land and water impacts of the AI boom," *Land Lines Magazine*, Lincoln Institute of Land Policy, Oct. 17, 2025.
- [17] V. Vaccaro, L. C. Tagliabue and M. Aldinucci, "Sustainable data centers: Advancing energy efficiency and resource optimization," in *Proc. 33rd Euromicro Int. Conf. Parallel, Distributed and Network-Based Processing (PDP)*, Turin, Italy, 2025, pp. 486–493.
- [18] A. Ahmed, D. Brown and A. Gegov, "Dynamic resource allocation through workload prediction for energy efficient computing," in *Advances in Computational Intelligence Systems*, P. Angelov, A. Gegov, C. Jayne and Q. Shen, Eds., Springer, 2017, pp. 35–44.